Planning Fast and Slow: Trajectory Synthesis via Conditional Flow Matching

Frank Tsu-Fang Lu

Master of Science in Computer Vision, Robotics and Machine Learning

from the

University of Surrey

School of Computer Science and Electrical and Electronic Engineering

Faculty of Engineering and Physical Sciences

University of Surrey

Guildford, Surrey, GU2 7XH, UK

September 2025

Supervised by: Dr. Simon Hadfield

DECLARATION OF ORIGINALITY

I confirm that the project dissertation I am submitting is entirely my own work and that any material used from other sources has been clearly identified and properly acknowledged and referenced. In submitting this final version of my report to the JISC anti-plagiarism software resource, I confirm that my work does not contravene the university regulations on plagiarism as described in the Student Handbook. In so doing I also acknowledge that I may be held to account for any particular instances of uncited work detected by the JISC anti-plagiarism software, or as may be found by the project examiner or project organiser. I also understand that if an allegation of plagiarism is upheld via an Academic Misconduct Hearing, then I may forfeit any credit for this module or a more severe penalty may be agreed.

MSc Dissertation Title: Guided Trajectory Synthesis via Flow Matching

Author Name: Frank Tsu-Fang Lu

Author Signature:  Date: September 2, 2025

Supervisor's name: Simon Hadfield

WORD COUNT

Number of Pages:

Number of Words:

ABSTRACT

This should provide a summary of what highlights the reader is going to expect to read. The length should be short, 200-300 words. An abstract should include: The significance of the work, which may require a brief overview of the topic area. The contributions that this work is making to the topic area, the key results that are presented Achievements to date, what they were and how they are paving the path for future work.

Note that this is the first thing that the examiner is going to read possibly after taking a glance at the title. They need to be informed about what they are going to expect to find in this report.

CONTENTS

Declaration of Originality ii

Word Count iii

Abstract iv

List of figures vii

1 Introduction 1

1.1 Background and Context 2

1.2 Objectives 3

1.3 Achievements 4

1.4 Overview of Dissertation 5

2 Background Theory and Literature Review 6

2.1 Offline Reinforcement Learning 6

2.1.1 Policy Constraint Methods 6

2.1.2 Implicit Policy Constraint Methods 7

2.1.3 Model-based Offline RL Methods 7

2.1.4 Conservative Q Learning 8

2.2 Diffusion Models 9

2.2.1 Diffusion's Roots in Variational Bayesian Methods 9

2.2.2 Deep Unsupervised Learning using Nonequilibrium Thermodynamics 10

2.2.3 Denoising Diffusion Probabilistic Models 12

2.3 Score-based Models 13

2.3.1 Denoising Score Matching 13

2.3.2 Generative Modeling by Estimating Gradients of the Data Distribution 14

2.3.3 Score-Based Generative Modeling Through Stochastic Differential Equations 14

2.4 Flow-based Models . . . 15
2.5 Planning with Diffusion . . . 15
2.6 Other Relevant Works . . . 15
2.7 Summary . . . 15

3 Flow Matching Methodologies 16
3.1 First Section . . . 16
3.1.1 First Subsection . . . 16
3.1.1.1 First Subsubsection . . . 16
3.1.2 Second Subsection . . . 16
3.2 Second Section . . . 16
3.3 Third Section . . . 17

4 Dataset Preparation 18
4.1 Section . . . 18
4.2 Section . . . 18

5 Experimental Results 19
5.1 Architectural Selection . . . 19
5.2 Modeling Joint Distribution . . . 19
5.3 Testing Different Conditioning Signals . . . 19
5.4 Hierarchical Planning . . . 19
5.4.1 Global Planner . . . 19
5.4.2 Local Planner . . . 19

6 Conclusions 20
6.1 Evaluation . . . 20
6.2 Future Work . . . 20

Bibliography 21

Appendix 25

LIST OF FIGURES

3.1 Example figure and caption. 17

1 INTRODUCTION

Imbuing intelligent agents with the capability for autonomous-decision-making in real-world environments is challenging. For decades, reinforcement learning has offered a robust mathematical framework for acquiring optimal behavioral skills through interactions within an environment to maximize cumulative rewards. The integration of deep neural networks as high-capacity function approximators has propelled RL even farther, leading to remarkable achievements across diverse domains, from mastering complex games to enabling basic robotic locomotion and manipulation.

However, the widespread success of modern machine learning, as seen in areas such as computer vision and natural language processing, fundamentally hinges on the availability of large and highly diverse datasets. In conventional, online RL, this data is collected through a continuous interaction loop: the agent executes its current policy in the environment, collects the resulting experiences into a buffer, and then uses this fresh data to update its policy. This cycle of exploration and optimization is repeated until the policy converges. With this paradigm, the acquisition of such extensive data becomes prohibitively impractical for real-world applications. For example, autonomous driving would need to practice in numerous cities for every model update, which is costly, time-consuming, let alone being unsafe. This inherent limitation created a gulf of generalization between lab-bound RL successes and the complex, open-world settings where intelligent decision-making is most critically needed.

Offline Reinforcement Learning emerges as a pivotal paradigm to bridge this gap. Offline RL focuses on enabling the training of policies solely from previously collected datasets, without additional online interaction. This approach holds immense potential for transforming vast amounts of historical data into robust, generalizable decision-making engines, much like how supervised learning revolutionized pattern recognition. The implications are profound across various fields from robotics, healthcare, and autonomous driving to logistics, finance, and operations research, as it offers a viable pathway to deploy sophisticated AI systems in domains where active data collection is unfeasible, costly, or unsafe.

Finally, the ability to leverage large and diverse datasets in an offline setting aligns the field of robotics learning with the broader paradigm of foundation models, as seen in computer vision and natural language processing. This opens the door to pre-training generalist agents on massive, heterogeneous datasets to acquire a broad repertoire of skills, which can then be efficiently fine-tuned for specific downstream tasks. This paradigm thus offers a compelling vision for the future: transforming vast archives of passive, observational data into active, embodied intelligence.

1.1 Background and Context

Despite its immense promise, offline RL is plagued by algorithmic challenges. At its core, offline RL is fundamentally about making and answering what Levine et al. describe as "counterfactual queries". These are "what if" questions about what might happen if the agent were to carry out actions different from those in the dataset. In other words, for an agent to improve upon the underlying behavior policy in the offline dataset, it must be able to execute action sequences that diverge from the dataset. This inherent need for counterfactual inference strains conventional machine learning tools, which assume that test data is drawn from the same distribution as training data, leading to a problematic distribution shift.

This issue is particularly acute for value-based methods. These methods learn a function to estimate the expected reward, or "goodness", of an action. When queried about an action that is far from the training data, this value estimate has no reliable data to ground its prediction. Function approximation errors can then cause the model to become unreliably optimistic, assigning erroneously high values to these unsupported actions [1]. The policy, trusting these flawed estimates, is then misguided into choosing brittle and unsafe behaviors when deployed.

This instability is particularly exacerbated by high-dimensional function approximators such as Deep Neural Networks (DNNs). The errors can accumulate over long horizons, making stable learning notoriously difficult. Furthermore, real-world datasets, especially those from human demonstrations, inherently exhibit multimodal action distributions: there are often multiple valid ways to perform a task. Traditional methods like behavior cloning struggle to capture this multimodality, often averaging across modes and leading to indecisive or jerky behaviors that fail to generalize [2].

To address the challenge of multimodality, generative modeling, particularly with denoising diffusion models, has emerged as a powerful approach. By framing planning as a sampling process, diffusion models can learn and represent complex, multi-modal distributions directly from data. This is a critical capability for robotics, where a task may have multiple valid solutions, such as different grasps for an object or various paths around an obstacle. By capturing the entire distribution of expert solutions, these models can enhance a robot's versatility and ability to generalize [2]-[4].

However, the very properties that make diffusion models powerful for generation introduce fundamental challenges for their deployment in real-time robotic systems. These models are built upon a stochastic, iterative framework that imposes a significant computational burden, leading to high-latency inference incompatible with dynamic control loops. More importantly, their stochastic nature is fundamentally misaligned with the deterministic physical laws and hard safety constraints that govern robotic motion, calling for an alternative generative paradigm.

Therefore, this dissertation presents Flow Planner, a hierarchical offline RL framework built upon flow matching, a powerful alternative that directly addresses the core limitations of diffusion models. Flow matching learns a deterministic transformation governed by an Ordinary Differential Equation (ODE) rather than a Stochastic Differential Equation (SDE). This fundamental distinction makes it not merely an incremental improvement but a paradigm inherently aligned with the deterministic, low-latency requirements of physical robotic systems [5]-[7].

1.2 Objectives

The primary aim of this dissertation is to develop and validate a novel, deterministic framework for imitation learning in offline reinforcement learning, leveraging the principles of flow matching. The specific objectives established to achieve this aim are as follows:

1. To design and validate a systematic methodology for curating offline reinforcement learning datasets from actively trained agents.
2. To design and implement "Flow Planner," a modular framework that utilizes flow matching for the generation of trajectories from offline data.

3. To analyze the impact of various conditioning mechanisms on the precision and generalizability of the trajectories generated by the framework.

4. To evaluate the performance of the ”Flow Planner” framework, benchmarking its core attributes such as inference speed and policy effectiveness against diffusion models.

1.3 Achievements

The primary contribution of this dissertation is the development and validation of Flow Planner, a novel hierarchical framework for imitation learning, which is composed of two independent technical achievements:

1. A Novel Formulation of Flow-Based Trajectory Planning: This work introduces the first specialization of the Conditional Flow Matching (CFM) framework for the domain of robotic trajectory planning. By reformulating the CFM objective to generate different sequences of states and actions, this contribution establishes a new class of deterministic, high-speed generative planners that directly address the latency limitations of contemporary diffusion-based methods.

2. A Hierarchical Framework for Long-Horizon Control: The second key contribution is the Flow Planner architecture, a new two-stage hierarchical framework that decouples long-horizon strategic reasoning from low-level reactive control. This design solves the scaling and coherence problems inherent in monolithic, end-to-end planners by factorizing the planning problem into a strategic Waypoint Planner and a reactive Action Planner, a novel application for flow-based models.

These core contributions are supported by:

1. A Comprehensive Analysis of Flow-Based Planners: This work provides an in-depth empirical analysis of applying flow-matching models to robotics, offering insights into the role of different model architectures and conditioning mechanisms, alongside a direct comparison to the diffusion paradigm.

2. A Reproducible Data Curation Pipeline and Benchmark Datasets: This work contributes a flexible and extensible pipeline for curating high-quality offline reinforcement learning datasets. This pipeline was used to generate three novel benchmark datasets, which have been publicly released alongside their corresponding trained expert agents to facilitate reproducibility and future research.

These contributions culminate in a robust and efficient framework for imitation learning, with the findings currently being prepared for submission to the IEEE International Conference on Intelligent Robots and Systems (IROS).

1.4 Overview of Dissertation

The remainder of this dissertation is structured as follows:

- Chapter 2 provides the theoretical foundation for this work, first examining established methods for addressing distribution shift problems, then exploring contemporary solutions that leverage generative models.
- Chapter 3 presents the core technical foundation of this dissertation. It details various approaches to dataset modeling with comprehensive mathematical formulations. The chapter ends by introducing "Flow Planner," the proposed hierarchical planning framework, and provides implementation specifics using Meta's flow matching library.
- Chapter 4 details the methodology for data curation and introduces the experimental environments. This includes a technical discussion on the implementation of the data pipeline using Gymnasium wrappers and the Minari library for dataset management.
- Chapter 5 provides a comprehensive analysis and discussion of the empirical results. This includes an investigation of various conditioning mechanisms and a benchmark comparison of the framework's performance against baseline models.
- Chapter 6 concludes the dissertation by summarizing the key contributions, discussing the implications of the findings, and outlining several promising directions for future research.

2 BACKGROUND THEORY AND LITERATURE REVIEW

This chapter begins by examining the fundamental concepts in offline reinforcement learning and established methodologies for addressing distributional shift. Next, it explores diffusion models through the lens of variational inference before unifying these concepts through continuous flows to introduce flow matching. The chapter concludes with an analysis of diffusion applications in trajectory synthesis and state-of-the-art approaches that integrate these models directly into reinforcement learning frameworks. Particular emphasis is placed on the mathematical formulations of diffusion and flow matching, as this constitutes the central focus of this dissertation and the applied methodologies after.

2.1 Offline Reinforcement Learning

As established in Chapter 1, a significant hurdle in offline reinforcement learning is the distributional shift between the static dataset and the policy being learned. This can lead to erroneous value estimates for out-of-distribution actions, resulting in brittle policies that perform poorly when deployed [1]. To mitigate this issue, the literature has broadly converged on three primary classes of approach: policy constraint methods, implicit policy constraint methods, and model-based methods.

2.1.1 Policy Constraint Methods

One early class of methods that's been explored extensively is policy constraint methods. These approaches aim to mitigate distributional shift by updating the policy not just by maximizing the Q-value, but by doing so subject to a constraint that the learned policy remains sufficiently close to the behavioral policy (collected data). This closeness is typically measured by a divergence metric, such as KL divergence or Maximum Mean Discrepancy (MMD).

To a degree, these methods can bound the distributional shift and the impact of out-of-distribution actions, potentially leading to more stable learning. Examples include BEAR (Bootstrapping Error Accumulation Reduction) by Kumar et al. [8] and BRAC (Behavior Regularized Actor Critic) by Wu et al. [9].

A significant challenge is their potential to be overly conservative, as a single constraint value ϵ might not work well across all states, especially if the behavior policy exhibits both highly random and highly deterministic actions. More critically, these methods often require accurately estimating the behavior policy itself, which can be extremely difficult, particularly when the data comes from diverse sources (e.g., humans or a mixture of different policies). Errors in this estimation can lead to an imperfect constraint and poor practical performance, making it a major bottleneck, especially during online fine-tuning.

2.1.2 Implicit Policy Constraint Methods

Building upon the insights into the limitations of explicit policy constraint methods, implicit policy constraint methods, such as AWAC (Advantage Weighted Actor Critic) by Nair et al. [10] , emerged as an alternative that addresses the difficulty of behavior policy modeling. AWAC leverages a theoretical identity that shows the solution to a constrained optimization problem can be approximated using a weighted maximum likelihood objective. In this formulation, actions taken from the behavior policy are weighted by the exponentiated advantage function, effectively embedding the constraint implicitly [10]. More intuitively, AWAC approximates the best new policy by look at actions already in the dataset and give more weight (advantage) to the really good ones.

This approach obviates the need for explicit modeling of the behavior policy, only requiring samples from it, which are readily available in the offline dataset. AWAC has shown to be effective for online fine-tuning after offline initialization and demonstrates strong performance on complex tasks like dexterous manipulation with human demonstration data.

While the sources highlight its advantages, it is still a form of policy constraint. Its inherent conservativeness might limit the extent to which it can discover entirely novel, optimal behaviors far from the observed data.

2.1.3 Model-based Offline RL Methods

Taking a different algorithmic direction, model-based offline RL methods, exemplified by MOPO (Model-based Offline Policy Optimization) by Yu et al. [11], propose to tackle the problem by learning a model of the environment's dynamics. While vanilla model-based RL (like MBPO) already showed some promise in offline settings, MOPO specifically mod-

ifies these algorithms to handle distributional shift. MOPO achieves this by punishing the policy for exploiting out-of-distribution states during model rollouts. This is implemented by subtracting an uncertainty penalty from the reward function, which becomes larger for state-action tuples that are more out of distribution (i.e., where the learned model's predictions are less certain).

MOPO hence has the ability to generalize beyond the state and action support of the offline data, enabling the learning of policies for tasks different from those for which the data was collected. It is more resilient to overestimation and overfitting than model-free methods and theoretically maximizes a lower bound of the policy's true return, balancing exploration and risk. Empirically, MOPO outperforms existing model-free offline RL methods and vanilla model-based approaches, especially on tasks requiring out-of-distribution generalization.

Nevertheless, models can still be inaccurate in regions far from the training data, and in a purely offline setting, these errors cannot be corrected through further interaction. The effectiveness relies heavily on the quality of the uncertainty estimation and the tuning of the penalty coefficient [12]. It may also struggle with datasets lacking sufficient action diversity, and there are limits to its generalization capabilities if the new task requires exploring states completely outside the available data support.

2.1.4 Conservative Q Learning

Finally, a distinct and highly effective approach is Conservative Q-Learning (CQL) by Kumar et al., which directly addresses the core problem of Q-value overestimation. Instead of constraining the policy, CQL augments the standard Bellman error objective with a novel Q-value regularizer. This regularizer works by finding and minimizing (pushing down) erroneously high Q-values for out-of-distribution actions, while a more advanced version also compensates by pushing up Q-values for actions present in the dataset. This makes the learned Q-function inherently "pessimistic" [13].

CQL is theoretically guaranteed to learn a lower bound on the true Q-function, effectively preventing the problematic overestimation. This direct approach leads to state-of-the-art performance across a wide range of tasks on standard benchmarks like D4RL and Atari. It significantly outperforms many previous offline RL methods, demonstrating

the ability to ”stitch” together parts of different trajectories to find more optimal paths, and achieving up to five times better results on challenging dexterous manipulation tasks. While conservative, the full CQL method only slightly underestimates Q-values, making it a robust and reliable algorithm.

The effectiveness of CQL depends on the appropriate selection of the regularization weight (alpha). Simpler variants of the algorithm without the compensatory term for in-distribution actions can lead to excessive underestimation. Despite its strong performance, ongoing research continues to explore further improvements.

In summary, policy constraint methods and implicit policy constraint methods both try to constrain the learned policy to remain close to the behavior policy. However, they do not aim to learn a pessimistic Q-function or ensure a lower bound on Q-values. They simply avoid the problematic evaluation of out-of-distribution actions. On the other hand, MOPO and CQL are methods that embrace a conservative and “pessimistic” strategy in order to mitigate the risks associated with going out of distributions and value overestimation by either directly pushing down the Q-values or penalized rewards based on uncertainty. This ensures the learned policies are robust, accepting the trade-off that may lead to sub-optimal but reliable performance.

2.2 Diffusion Models

The following two sections provide the necessary background theory on diffusion and score-based models preceding flow matching. Section 2.2 introduces the formulation of diffusion models, one derived from a variational inference perspective. Section 2.3 explores a more general score-based approach defined by continuous-time stochastic differential equations. Finally, section 2.4 introduces flow matching, a simulation-free method for training Continuous Normalizing Flows based on deterministic Ordinary Differential Equations (ODEs).

2.2.1 Diffusion’s Roots in Variational Bayesian Methods

In probabilistic generative modeling, the common practice is to introduce latent variables z and a parameterized generative model $p_\theta(x)$ with prior $p(z)$ to explain observations x. By Bayes’ rule, the model’s posterior distribution is expressed as:

$$p_\theta(z \mid x) = \frac{p_\theta(x \mid z)p(z)}{p_\theta(x)}.$$

Where the denominator is the marginal likelihood or evidence, calculated by

$$p_\theta(x) = \int p_\theta(x \mid z)\, p(z)\, dz.$$

The posterior $p(z \mid x)$ explains how likely different latent configurations are given the data, but computing it directly is intractable because calculating the evidence requires integrating over latent space, which explodes in complexity. This intractability means we cannot directly evaluate or differentiate $p(x)$.

Hence, the Bayesian objective is maximizing the marginal likelihood, or evidence of the observed data $\log p_\theta(x)$. It quantifies how well a generative model explains the data overall, marginalizing out the latent space. Variational Bayesian methods provide a practical workaround by introducing a tractable approximation $q_\phi(z \mid x)$, referred to as a recognition model or probabilistic encoder, to the true posterior and maximizing the evidence lower bound (ELBO) [14]:

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)}\Big[\log p_\theta(x, z) - \log q_\phi(z \mid x)\Big],$$

or more explicitly,

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)}\Big[\log p_\theta(x \mid z) + \log p(z) - \log q_\phi(z \mid x)\Big].$$

2.2.2 Deep Unsupervised Learning using Nonequilibrium Thermodynamics

Building on principles from non-equilibrium thermodynamics, diffusion probabilistic models systematically and slowly destroy data structure by applying an iterative forward diffusion process that gradually perturbs data $x_0 \sim p_{\text{data}}(x)$ into a tractable Gaussian noise via a Markov chain:

$$q(x_{1:T} \mid x_0) = \prod_{t=1}^{T} q(x_t \mid x_{t-1}).$$

This interestingly mirrors physical diffusion processes such as ink dispersing in water or smoke spreading in air. This forward process spreads probability mass across the data landscape, creating training signals far from the original data and grounding the method in the same mathematics as Brownian motion.

A reverse diffusion process is then learned to restore this structure. The ideal reverse process $q(x_{t-1} \mid x_t)$, as mentioned in 2.2.1, is intractable as it depends on the entire data distribution. Dickstein et al. therefore parameterized an approximation p_θ designed to match the tractable posterior $q(x_{t-1} \mid x_t, x_0)$ [15]:

$$p_\theta(x_{t-1} \mid x_t) \;\approx\; q(x_{t-1} \mid x_t, x_0).$$

The trained model serves as a local compass that estimates the gradient of the log probability landscape $\nabla_x \log p_t(x)$, which points towards regions of higher data likelihood. Here, $p_t(x)$ denotes the time-varying marginal distribution of data at diffusion step t, obtained by evolving clean samples $x_0 \sim p_{data}$ through the forward noising process. This model does not learn $p_t(x)$ directly, but instead estimate its score function. This connects diffusion to score matching later covered extensively in Section 2.3.

Adapting variational inference, one can derive a variational lower bound on this log-likelihood:

$$\log p_\theta(x_0) \;\geq\; \mathbb{E}_{q(x_{0:T})}\Big[\log p_\theta(x_{0:T}) - \log q(x_{1:T} \mid x_0)\Big].$$

This entire framework can be understood intuitively through the lens of variational inference.

- The fixed forward noising process $q(x_{1:T} \mid x_0)$ serves as the probabilistic encoder, mapping the data x_0 into a latent space of noisy variables $x_{1:T}$. - The learned reverse

process $p_\theta(x_{t-1} \mid x_t)$ acts as the probabilistic decoder, which is trained to reverse the noising process.

This means that training diffusion models amounts to optimizing a diffusion-specific ELBO, which remarkably simplifies to training the decoder to predict the noise added at each step, as demonstrated in the subsequent section.

2.2.3 Denoising Diffusion Probabilistic Models

While earlier works established the connection between diffusion and variational inference, the objective was still complex. Ho et al.'s significantly popularized the framework by reparameterizing the model's learning target [16]. The reverse process learns the transition

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}\big(x_{t-1};\, \mu_\theta(x_t, t),\, \Sigma_\theta(x_t, t)\big),$$

which is modeled as a Gaussian with a mean $\mu_\theta(x_t, t)$ and variance $\Sigma_\theta(x_t, t)$. The DDPM authors made two key simplifications:

First, they found that fixing the variance to a known constant was sufficient for high-quality results. Second, and most importantly, they reparameterized the mean. Instead of training the network to directly predict the mean of the reverse step, they trained it to predict the noise, ϵ, that was originally added to the clean data.

This transforms the complex variational objective into a simple mean-squared error loss between the true noise and the predicted noise:

$$\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{t, x_0, \epsilon}\big[\, \|\epsilon - \epsilon_\theta(x_t, t)\|^2 \,\big],$$

where ϵ is the true Gaussian noise sampled at a random timestep t, and $\epsilon_\theta(x_t, t)$ is the network's prediction given the noisy image x_t and timestep t. This simplified objective was not only more stable to train but also produced significantly higher-quality samples, which was the major breakthrough that led to the widespread adoption of diffusion models.

2.3 Score-based Models

An alternative and powerful approach to generative modeling is to learn the score function of the data distribution, which is formally defined as the gradient of its log probability $\nabla_x \log p_{\text{data}}(x)$ [15]. This score function creates a vector field that points towards regions of higher data density, providing a trajectory back to the data manifold from any point in the space. Once this score field is learned, new samples can be generated by starting from random noise and iteratively moving in the direction of the score, a process known as Langevin dynamics.

2.3.1 Denoising Score Matching

Estimating the score function directly is difficult. Vincent et al. provided a workaround in Denoising Score Matching (DSM). The key insight was that the objective of training a simple network to denoise data corrupted with Gaussian noise is equivalent to learning the score of that noise-perturbed data distribution [18]. Given a clean sample x, corruption is defined as

$$\tilde{x} = x + \sigma\epsilon, \quad \epsilon \sim \mathcal{N}(0, I),$$

$\tilde{x}$ being a noisy sample. When the corruption is additive isotropic Gaussian noise, the target score $\nabla_{\tilde{x}} \log q_\sigma(\tilde{x}|x)$ for a noisy sample $\tilde{x}$ given a clean sample x simplifies to $\frac{1}{\sigma^2}(x - \tilde{x})$.

$$\nabla_{\tilde{x}} \log q_\sigma(\tilde{x} \mid x) = \frac{1}{\sigma^2}(x - \tilde{x}) = -\frac{\epsilon}{\sigma}.$$

This shows that the target score is directly proportional to the negative noise. This is why instead of a complex score matching objective, one could simply train a denoising network $D_\theta(\tilde{x})$ with a straightforward mean-squared error loss like the one in DDPM:

$$\mathcal{L}_{\text{DSM}}(\theta) = \mathbb{E}_{x,\tilde{x}}\Big[\, \|D_\theta(\tilde{x}) - x\|^2 \,\Big].$$

This made score estimation practical and stable.

2.3.2 Generative Modeling by Estimating Gradients of the Data Distribution

While DSM worked for a single noise level, it struggled with real-world data that often resides on a low-dimensional manifold, leaving the score undefined in most of the space. Song et al. solved this problem similar to diffusion by perturbing the data with multiple levels of Gaussian noise. They then trained a single Noise Conditional Score Network (NCSN), denoted $s_\theta(x_t, t)$, to jointly estimate the scores for all the different noise-perturbed distributions. This effectively creates a well-defined vector field at all locations and scales [19]. For generation, annealed Langevin dynamics is introduced. The process starts with pure Gaussian noise and follows the gradient provided by the learned score network s_θ. The noise level σ is slowly annealed from high to low, guiding the sample from the simple noise distribution back to the complex data distribution via an iterative update step:

$$x_{t-1} = x_t + \alpha s_\theta(x_t, \sigma_t) + \sqrt{2\alpha} z_t$$

where α is the step size and $z_t \sim \mathcal{N}(0, I)$ is a standard Gaussian noise term.

2.3.3 Score-Based Generative Modeling Through Stochastic Differential Equations

The SDE framework unifies score matching and DDPMs by describing them as discretizations of a single continuous-time process.The forward SDE is defined to transform a complex data distribution into a simple noise prior over a time interval $t \in [0, T]$:

$$dx = \underbrace{f(x,t)}_{\text{drift}}\, dt + \underbrace{g(x,t)}_{\text{diffusion}}\, dW,$$

where W is a standard Wiener process and $f(x,t)$ and $g(x,t)$ are the drift and diffusion coefficients, respectively. This describes a continuum of noisy intermediate distributions $\{p_t(x)\}$. Additionally, a corresponding reverse-time SDE that can reverse this noising process exists:

$$dx = \underbrace{\left[f(x,t) - g(x,t)^2 \nabla_x \log p_t(x)\right]}_{\text{drift}}\, dt + \underbrace{g(x,t)}_{\text{diffusion}}\, d\bar{W},$$

where $d\bar{W}$ is a standard Wiener process run backward in time. Much like the forward SDE, one can intuitively break this equation down into a deterministic drift term and

a stochastic diffusion term. And conveniently, the deterministic dynamics depend only on the score function $\nabla_x \log p_t(x)$. Note that for models like DDPMs, the diffusion term simplifies from the general form $g(x,t)$ to $g(t)$ because the noise schedule is designed to be state-independent, as detailed further in Appendix A. Since this score is intractable, it is approximated by a neural network $s_\theta(x,t)$, which can then be used by numerical solvers to generate samples [19]. This mathematically elegant framework unlocks some new capabilities:

1. Flexible sampling: Because the process is continuous, practitioners are no longer locked into a fixed number of discrete steps. This allows for a direct trade-off between computation speed and sample quality, as any numerical SDE solver can be used to generate samples.

2. The continuous nature of the SDE makes it highly amenable to controllable generation. The reverse sampling process can be guided to solve inverse problems like image inpainting and colorization, often without needing to retrain the model.

3. Exact Log-Likelihoods via an Equivalent ODE: The SDE framework revealed a corresponding deterministic process, described by a neural ordinary differential equation (ODE) known as the probability flow ODE [20]. This breakthrough not only enables deterministic sampling and exact likelihood computation but also provides the direct theoretical inspiration for Flow Matching, which will be detailed in the next section.

2.4 Flow-based Models

2.5 Planning with Diffusion

2.6 Other Relevant Works

Note that references work like this [?] [?]

2.7 Summary

Your final section should include a summary of the chapter, which will summarise what has been covered and where there are gaps in knowledge which pave the way into the original contributions you make in the coming chapters.

3 FLOW MATCHING METHODOLOGIES

The technical body of the report consists of a number of chapters (just one here, but there will usually be more) of work you have carried out to date. Follow a logical structure in how you present your work. This will usually be the phases of the work you carried out which will link loosely to the objectives. Note that these chapters are not meant to be a log book, thus do not write them like a diary of work carried out but as a technical report specifying clearly what has been implemented and presented in a way that someone else could carry out the work based on what they have read.

Again you need to consult with your supervisor on the technical chapters you plan to write by providing a draft contents page. It should show clearly the sections in it and every chapter should have a clear paragraph introducing it and what to expect in it.

3.1 First Section

Structure your text into sections.

3.1.1 First Subsection

If necessary, also use subsections. Subsections are entered using the Heading 3 paragraph style (all these heading styles are self-numbering).

3.1.1.1 First Subsubsection

If you really need subsubsections.

3.1.2 Second Subsection

And, as required, more subsections.

3.2 Second Section

As an example of a figure, consider Figure 3.1 and note how it is referenced here by cross referencing to a caption method so that also you can form a list of figures at the start. You should be familiar with how to insert a caption and refer to it in Word from the training needs analysis form you were given. The caption should include title and essential

information about figure and figures must always be referred to in the text. Ensure you insert figures into appropriate points in the text so they are both near to the text that is referring to the figure but also ensure that they are arranged in a tidy fashion so they do not leave big gaps with no text.

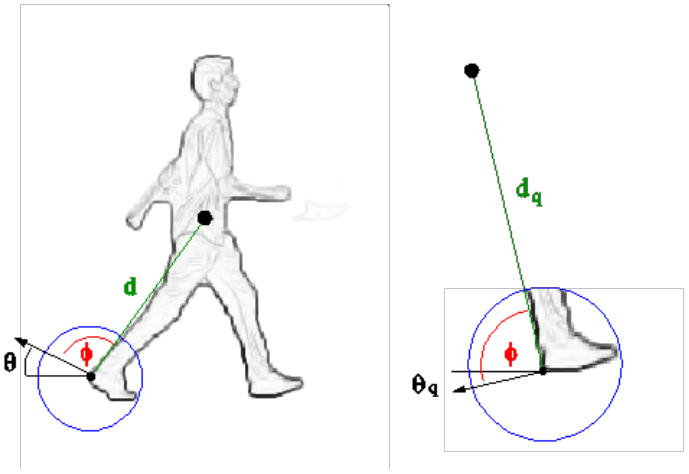

Figure 3.1: Example figure and caption.

Table 3.1: Make sure to set caption to large size.

Test	Test
More	Testing
More	Testing

3.3 Third Section

This sections hows how to do equations and to show that they number correctly as LaTeX can do with the chapter number and equation number to refer to equation in a piece of text:

$$y = x^2 \tag{3.1}$$

but note it is usually after putting in an equation that you may refer back to equation (3.1) later in the document or chapter.

4 DATASET PREPARATION

You need to work out how many technical chapters you will require, possibly more than two in this template. Normally your final chapter before the conclusion will include some clear results or proof of concepts from the end goals of your dissertation and it is important to document these well and show the value of what you have achieved. Frequently many dissertations fail to document this information correctly because it is rushed at the end so do ensure you allow time to write it properly.

4.1 Section

Text to insert

4.2 Section

Text to insert – but ensure the last section really does show the value of your work and where you have got to.

5 EXPERIMENTAL RESULTS

5.1 Architectural Selection

5.2 Modeling Joint Distribution

5.3 Testing Different Conditioning Signals

5.4 Hierarchical Planning

5.4.1 Global Planner

5.4.2 Local Planner

6 CONCLUSIONS

Your conclusion should summarise what you have carried out in this work to date, what you have achieved and what you are on course to achieve. Also it should explain what was documented in this dissertation. Note that someone may read this chapter without bothering to read the earlier chapters so there is no problem with repeating what you may have already written earlier. Indeed no new information should be in this part of the conclusion. It should reference back to what is already there.

6.1 Evaluation

Stand back and evaluate what you have achieved and how well you have met the objectives. Evaluate your achievements against your objectives in section 1.2. Demonstrate that you have tackled the project in a professional manner.

6.2 Future Work

Discuss what further work could be carried out from this work and relate it to what has been carried out.

BIBLIOGRAPHY

[1] Black, K. , Brown, N. , Driess, D. , Esmail, A. , Equi, M. , Finn, C. , Fusai, N. , Groom, L. , Hausman, K. , Ichter, B. , Jakubczak, S. , Jones, T. , Ke, L. , Levine, S. , Li-Bell, A. , Mothukuri, M. , Nair, S. , Pertsch, K. , Shi, L. X. , Tanner, J. , Vuong, Q. , Walling, A. , Wang, H. , and Zhilinsky, U. . $π_0$: A Vision-Language-Action Flow Model for General Robot Control, Nov. 2024.

[2] Chen, R. T. Q. , Rubanova, Y. , Bettencourt, J. , and Duvenaud, D. . Neural Ordinary Differential Equations, Dec. 2019.

[3] Chi, C. , Feng, S. , Du, Y. , Xu, Z. , Cousineau, E. , Burchfiel, B. , and Song, S. . Diffusion Policy: Visuomotor Policy Learning via Action Diffusion, June 2023.

[4] Dinh, L. , Sohl-Dickstein, J. , and Bengio, S. . Density estimation using Real NVP, Feb. 2017.

[5] Fan, W. , Zheng, A. Y. , Yeh, R. A. , and Liu, Z. . CFG-Zero*: Improved Classifier-Free Guidance for Flow Matching Models, Apr. 2025.

[6] Florence, P. , Lynch, C. , Zeng, A. , Ramirez, O. , Wahid, A. , Downs, L. , Wong, A. , Lee, J. , Mordatch, I. , and Tompson, J. . Implicit Behavioral Cloning, Sept. 2021.

[7] Gao, Y. , Huang, J. , Jiao, Y. , and Zheng, S. . Convergence of Continuous Normalizing Flows for Learning Probability Distributions, Mar. 2024.

[8] Geng, Z. , Deng, M. , Bai, X. , Kolter, J. Z. , and He, K. . Mean Flows for One-step Generative Modeling, May 2025.

[9] Grathwohl, W. , Chen, R. T. Q. , Bettencourt, J. , Sutskever, I. , and Duvenaud, D. . FFJORD: Free-form Continuous Dynamics for Scalable Reversible Generative Models, Oct. 2018.

[10] Ha, D. and Schmidhuber, J. . World Models. Mar. 2018. doi: 10.5281/zenodo.1207631.

[11] Ho, J. , Jain, A. , and Abbeel, P. . Denoising Diffusion Probabilistic Models, Dec. 2020.

[12] Holderrieth, P. and Erives, E. . An Introduction to Flow Matching and Diffusion Models, June 2025.

[13] Hyvarinen, A. . Estimation of Non-Normalized Statistical Models by Score Matching.

[14] Janner, M. , Fu, J. , Zhang, M. , and Levine, S. . When to Trust Your Model: Model-Based Policy Optimization, Nov. 2021.

[15] Janner, M. , Du, Y. , Tenenbaum, J. B. , and Levine, S. . Planning with Diffusion for Flexible Behavior Synthesis, Dec. 2022.

[16] Kim, M. J. , Pertsch, K. , Karamcheti, S. , Xiao, T. , Balakrishna, A. , Nair, S. , Rafailov, R. , Foster, E. , Lam, G. , Sanketi, P. , Vuong, Q. , Kollar, T. , Burchfiel, B. , Tedrake, R. , Sadigh, D. , Levine, S. , Liang, P. , and Finn, C. . OpenVLA: An Open-Source Vision-Language-Action Model, Sept. 2024.

[17] Kingma, D. P. and Dhariwal, P. . Glow: Generative Flow with Invertible 1x1 Convolutions, July 2018.

[18] Kingma, D. P. and Welling, M. . Auto-Encoding Variational Bayes, Dec. 2022.

[19] Kostrikov, I. , Nair, A. , and Levine, S. . Offline Reinforcement Learning with Implicit Q-Learning, Oct. 2021.

[20] Kumar, A. , Fu, J. , Tucker, G. , and Levine, S. . Stabilizing Off-Policy Q-Learning via Bootstrapping Error Reduction, Nov. 2019.

[21] Kumar, A. , Zhou, A. , Tucker, G. , and Levine, S. . Conservative Q-Learning for Offline Reinforcement Learning, Aug. 2020.

[22] Levine, S. , Kumar, A. , Tucker, G. , and Fu, J. . Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems, Nov. 2020.

[23] Lipman, Y. , Chen, R. T. Q. , Ben-Hamu, H. , Nickel, M. , and Le, M. . Flow Matching for Generative Modeling, Feb. 2023.

[24] Lipman, Y. , Havasi, M. , Holderrieth, P. , Shaul, N. , Le, M. , Karrer, B. , Chen, R. T. Q. , Lopez-Paz, D. , Ben-Hamu, H. , and Gat, I. . Flow Matching Guide and Code, Dec. 2024.

[25] Liu, X. , Gong, C. , and Liu, Q. . Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow, Sept. 2022.

[26] Ma, N. , Tong, S. , Jia, H. , Hu, H. , Su, Y.-C. , Zhang, M. , Yang, X. , Li, Y. , Jaakkola, T. , Jia, X. , and Xie, S. . Inference-Time Scaling for Diffusion Models beyond Scaling Denoising Steps, Jan. 2025.

[27] Mayne, D. Q. , Rawlings, J. B. , Rao, C. V. , and Scokaert, P. O. M. . Constrained model predictive control: Stability and optimality. Automatica, 36(6):789–814, June 2000. ISSN 0005-1098. doi: 10.1016/S0005-1098(99)00214-9.

[28] Nagabandi, A. , Kahn, G. , Fearing, R. S. , and Levine, S. . Neural Network Dynamics for Model-Based Deep Reinforcement Learning with Model-Free Fine-Tuning, Dec. 2017.

[29] Nair, A. , Gupta, A. , Dalal, M. , and Levine, S. . AWAC: Accelerating Online Reinforcement Learning with Offline Datasets, Apr. 2021.

[30] Papamakarios, G. , Nalisnick, E. , Rezende, D. J. , Mohamed, S. , and Lakshminarayanan, B. . Normalizing Flows for Probabilistic Modeling and Inference.

[31] Park, S. , Frans, K. , Eysenbach, B. , and Levine, S. . OGBench: Benchmarking Offline Goal-Conditioned RL, Feb. 2025.

[32] Park, S. , Li, Q. , and Levine, S. . Flow Q-Learning, May 2025.

[33] Pathak, D. , Agrawal, P. , Efros, A. A. , and Darrell, T. . Curiosity-driven Exploration by Self-supervised Prediction, May 2017.

[34] Psenka, M. , Escontrela, A. , Abbeel, P. , and Ma, Y. . Learning a Diffusion Model Policy from Rewards via Q-Score Matching, Feb. 2025.

[35] Schulman, J. , Wolski, F. , Dhariwal, P. , Radford, A. , and Klimov, O. . Proximal Policy Optimization Algorithms, Aug. 2017.

[36] Sohl-Dickstein, J. , Weiss, E. A. , Maheswaranathan, N. , and Ganguli, S. . Deep Unsupervised Learning using Nonequilibrium Thermodynamics, Nov. 2015.

[37] Song, Y. and Ermon, S. . Generative Modeling by Estimating Gradients of the Data Distribution, Oct. 2020.

[38] Song, Y. , Sohl-Dickstein, J. , Kingma, D. P. , Kumar, A. , Ermon, S. , and Poole, B. . Score-Based Generative Modeling through Stochastic Differential Equations, Feb. 2021.

[39] Sutton, R. S. and Barto, A. G. . Reinforcement Learning: An Introduction.

[40] Tong, A. , Malkin, N. , Fatras, K. , Atanackovic, L. , Zhang, Y. , Huguet, G. , Wolf, G. , and Bengio, Y. . Simulation-free Schrödinger bridges via score and flow matching, Mar. 2024.

[41] Vincent, P. . A Connection Between Score Matching and Denoising Autoencoders. Neural Computation, 23(7):1661–1674, July 2011. ISSN 0899-7667, 1530-888X. doi: 10.1162/NECO_a_00142.

[42] Wagenmaker, A. , Nakamoto, M. , Zhang, Y. , Park, S. , Yagoub, W. , Nagabandi, A. , Gupta, A. , and Levine, S. . Steering Your Diffusion Policy with Latent Space Reinforcement Learning, June 2025.

[43] Wang, G. , Li, J. , Sun, Y. , Chen, X. , Liu, C. , Wu, Y. , Lu, M. , Song, S. , and Yadkori, Y. A. . Hierarchical Reasoning Model, Aug. 2025.

[44] Wang, Z. , Hunt, J. J. , and Zhou, M. . Diffusion Policies as an Expressive Policy Class for Offline Reinforcement Learning, Aug. 2023.

[45] Wu, Y. , Tucker, G. , and Nachum, O. . Behavior Regularized Offline Reinforcement Learning, Nov. 2019.

[46] Yıldız, Ç. , Heinonen, M. , and Lähdesmäki, H. . Continuous-Time Model-Based Reinforcement Learning, June 2021.

[47] Yu, T. , Thomas, G. , Yu, L. , Ermon, S. , Zou, J. , Levine, S. , Finn, C. , and Ma, T. . MOPO: Model-based Offline Policy Optimization, Nov. 2020.

[48] Zhou, M. , Zheng, H. , Wang, Z. , Yin, M. , and Huang, H. . Score identity Distillation: Exponentially Fast Distillation of Pretrained Diffusion Models for One-Step Generation, May 2024.

[49] Zhou, M. , Gu, Y. , and Wang, Z. . Few-Step Diffusion via Score identity Distillation, May 2025.

APPENDIX

You may have one or more appendices containing detail, bulky or reference material that is relevant though supplementary to the main text: perhaps additional specifications, tables or diagrams that would distract the reader if placed in the main part of the dissertation. Make sure that you place appropriate cross-references in the main text to direct the reader to the relevant appendices. Again you should discuss with your supervisor what appendices are appropriate for this work.

Note that you should not include your program listings as an appendix or appendices. You should submit one copy of such bulky text as a separate item, perhaps on a disk.

A.1 Derivation of the Conditional Flow Matching Objective